



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

**Tema 100. La migración de aplicaciones en el marco de
procesos de ajuste dimensional y por obsolescencia técnica.
Gestión de la configuración y de versiones. Gestión de
entornos**

AUTOR: Marcos Martínez Díaz

Actualización 2017

ÍNDICE

1. INTRODUCCIÓN	3
2. LA MIGRACIÓN DE APLICACIONES EN EL MARCO DE PROCESOS DE AJUSTE DIMENSIONAL Y POR OBSOLESCENCIA	4
2.1 AJUSTE DIMENSIONAL	4
2.2 DOWNSIZING, UPSIZING Y RIGTHSIZING	4
2.2.1 Downsizing	4
2.2.2 Upsizing	5
2.2.3 Rightsizing	5
2.3 OBSOLESCENCIA DE APLICACIONES Y MIGRACIÓN	5
2.4 TIPOS DE MIGRACIÓN DE APLICACIONES	5
2.5 CENATIC – MIGRACIÓN DE APLICACIONES A SOFTWARE DE FUENTES ABIERTAS	6
3. GESTIÓN DE LA CONFIGURACIÓN	7
3.1 MÉTRICA V3 – INTERFAZ GC	7
3.1.1 Estudio de Viabilidad del Sistema (EVS)	7
3.1.2 Análisis, diseño, construcción e implantación y aceptación del sistema de información	8
3.1.3 Mantenimiento del Sistema de Información (MSI)	9
3.2 ITIL – SERVICE AND ASSET CONFIGURATION MANAGEMENT	9
3.2.1 Elementos clave de SACM	9
3.2.2 El proceso de SACM	10
3.3 GESTIÓN DE LA CONFIGURACIÓN EN ESTÁNDARES Y METODOLOGÍAS	10
4. GESTIÓN DE VERSIONES	12
4.1 ITIL – GESTIÓN DE ENTREGAS Y DESPLIEGUES	12
4.2 GESTIÓN DE VERSIONES Y RELEASES EN EL DESARROLLO SOFTWARE	13
4.2.1 Terminología común en la gestión de versiones	13
4.2.2 Integración continua	13
4.3 HERRAMIENTAS PARA LA GESTIÓN DE LA CONFIGURACIÓN	14
4.3.1 Herramientas para la gestión de la configuración orientadas al gobierno de los sistemas de información	14
4.3.2 Herramientas para la gestión de versiones y la configuración en proyectos de desarrollo ..	14
5. GESTIÓN DE ENTORNOS	16
PREGUNTAS DE TEST	19
SOLUCIONES A LAS PREGUNTAS DE TEST	22
BIBLIOGRAFÍA BÁSICA	23
BIBLIOGRAFÍA PARA AMPLIAR EL TEMA	23

1. Introducción

El presente tema está estructurado en cuatro grandes bloques, siguiendo su título. En primer lugar se abordará la migración de aplicaciones en el marco de procesos de ajuste dimensional y por obsolescencia técnica, posteriormente la gestión de la configuración y versiones y, finalmente, la gestión de entornos.

El estudio del bloque de migración (apartado 2) puede realizarse de forma independiente. Para los apartados relativos a la gestión de la configuración, versiones y entorno (apartados 3 al 5) se recomienda estudiarlos conjuntamente dadas las relaciones entre los mismos.

Se ha pretendido facilitar al opositor una visión completa y esquemática de ambos temas, por lo que la redacción del tema se ha realizado de forma escueta, centrándose en los conceptos clave, resaltando en letra negrita términos relevantes.

2. La migración de aplicaciones en el marco de procesos de ajuste dimensional y por obsolescencia

La migración de aplicaciones entre plataformas diferentes es una tarea que puede realizarse por diferentes causas. En el caso del ajuste dimensional, el principal motivo es pasar de arquitecturas *mainframe* a modelos más económicos y abiertos. En el caso de la obsolescencia, la causa de la migración puede ser la obsolescencia técnica de la plataforma *hardware*, la obsolescencia del lenguaje de programación, de la funcionalidad de la propia aplicación o de su arquitectura, entre otros.

2.1 Ajuste dimensional

El ajuste dimensional, consiste en aligerar la carga de trabajo que soporta un gran ordenador (*mainframe*) trasladándola a otro tipo de ordenadores de menor coste. Suele conllevar una redistribución de recursos de proceso, acercándolos a los usuarios finales o distribuyéndolos en otras arquitecturas, como cliente-servidor. Es, en realidad, un concepto que tuvo su mayor apogeo hace varias décadas cuando las arquitecturas *mainframe* eran más populares que hoy en día y fueron migradas en diversas organizaciones a entornos de computación distribuidos.

2.2 Downsizing, upsizing y rightsizing

El *downsizing* aparece a principios de los años 80, y luego, como resultado de esta primera generación aparece el *rightsizing* en los 90. Surgen como resultado de la presión por reducir costes TIC, así como de una gran demanda de aplicaciones, por parte de las organizaciones en las que el entorno *mainframe* era popular.

2.2.1 Downsizing

Se entiende como la sustitución, total o parcial, del centro de cómputo centralizado de una organización, por una red de servidores/ordenadores más pequeños, económicos y versátiles. Es el proceso de mover una aplicación o parte de ella de un *mainframe* a una plataforma menos costosa.

Hoy en día el número de fabricantes de sistemas *mainframe* es reducido y sus costes de adquisición de *hardware* son, en general, altos en comparación con sistemas de mayor difusión (servidores convencionales, etc.). Adicionalmente, existe un volumen mucho mayor de recursos humanos en el mercado con conocimientos de entornos populares como Linux y Microsoft que de entornos *mainframe*.

Ventajas:

1. No hay dependencia de un solo sistema (*mainframe*).
2. Portabilidad, dado que se suelen emplear arquitecturas abiertas.
3. Interoperabilidad entre distintos sistemas.
4. Reducción considerable de costes operativos y costes de adquisición y/o actualización como de mantenimiento de infraestructura TIC.
5. Posibilidad de optimizar y balancear la utilización del *hardware* y *software* existente
6. Facilidad para implementar medidas orientadas a la alta disponibilidad de la información.
7. Mayor flexibilidad.
8. Reducción (o, por lo menos, redistribución) de espacio físico requerido.

Desventajas:

1. Posibilidad de carencias funcionales si la migración no se realiza correctamente.
2. Mayor variedad y heterogeneidad de recursos en máquinas, equipamiento y aplicaciones.
3. Mayor necesidad de monitorización, supervisión, actualización de servidores, al estar distribuidos.

2.2.2 Upsizing

Se utiliza el término *upsizing* para referirse a la integración en entornos de red de aplicaciones y ordenadores que estaban aislados, de forma que se permita la compartición de datos. Un ejemplo sería la integración de bases de datos aisladas en un servidor único de base de datos.

El *downsizing* de aplicaciones desde grandes ordenadores a servidores de menor envergadura requiere un sistema de red que combine la seguridad y las capacidades multitarea con flexibilidad para adaptarse a entornos informáticos variados. Se necesitan unos requisitos similares para integrar (*upsizing*) aplicaciones aisladas con el fin de utilizarlas de manera conjunta aprovechando así su facilidad de uso y su productividad.

2.2.3 Rightsizing

Se refiere a la elección de la plataforma más apropiada para las actividades informáticas de una organización. Es un término que surge en contraste con el *downsizing* y *upsizing*, refiriéndose a que la elección de una plataforma variará según las necesidades particulares de una organización. Por lo tanto, para determinados requisitos, puede ser apropiado utilizar un súper-ordenador o un *mainframe*, y para otros requerimientos, será recomendable una arquitectura cliente-servidor.

2.3 Obsolescencia de aplicaciones y migración

Las aplicaciones obsoletas o con riesgo de obsolescencia suelen denominarse aplicaciones *legacy*, legadas, o heredadas. Existen diversos motivos, además de la obsolescencia de la propia aplicación, por lo que puede ser necesario realizar una migración de aplicaciones por obsolescencia. A continuación se citan algunos:

1. **Obsolescencia hardware:** la plataforma sobre la que se ejecuta la aplicación no puede ser actualizada o ampliada por limitaciones técnicas, por lo que es necesario sustituirla.
2. **Final de soporte:** el fabricante del sistema (*hardware* o *software*) deja de proporcionar soporte por motivos de obsolescencia (es lo conocido como **end-of-life** o **EOL**). Igualmente, puede dejar de existir soporte porque el fabricante cierra o cesa su actividad.
3. **Código fuente no adaptable:** el código fuente de la aplicación puede no ser adaptable a las nuevas necesidades, por ejemplo si se trata de acceso web, movilidad, etc.
4. **Complejidad del código y software rot (código podrido):** especialmente en el caso de desarrollos a medida, el código de la aplicación puede volverse excesivamente complejo con el paso del tiempo si no se ha mantenido con criterios de modularidad, documentación, etc. Es lo conocido en algunos casos como “código espagueti”. El mantenimiento o modificación de la aplicación puede volverse tan complejo y costoso que sea necesario una migración de la aplicación.
5. **Reducción de costes:** si la aplicación en uso es propietaria, puede resultar beneficioso en términos económicos migrarla a *software* libre, con el fin de eliminar los costes derivados de las licencias.

2.4 Tipos de migración de aplicaciones

La migración de aplicaciones puede realizarse en diferentes modalidades:

- **Refronting o refacing:** consiste en modificar únicamente la interfaz gráfica de una aplicación, sin necesidad de reescribir la aplicación completa. Puede modificarse la entrada de datos, o la forma en la que se presenta la aplicación, por ejemplo pasando de interfaces nativos de la aplicación a interfaces web.
- **Replacement (sustitución):** en esta modalidad se sustituyen las aplicaciones completas por una o varias aplicaciones.
- **Rehosting (re-alojamiento):** consiste en mover la aplicación desde un entorno *hardware legacy* a un entorno más moderno, sin modificar el código de la aplicación o su arquitectura.
- **Rearchitecting (re-arquitectura o reescritura):** consiste en desarrollar de nuevo la aplicación utilizando nuevos paradigmas de programación (por ejemplo desarrollo web, orientación a objetos, u otros). Esta modalidad puede requerir el empleo de técnicas de reingeniería e ingeniería inversa para determinar la funcionalidad completa de la aplicación antes de migrarla.

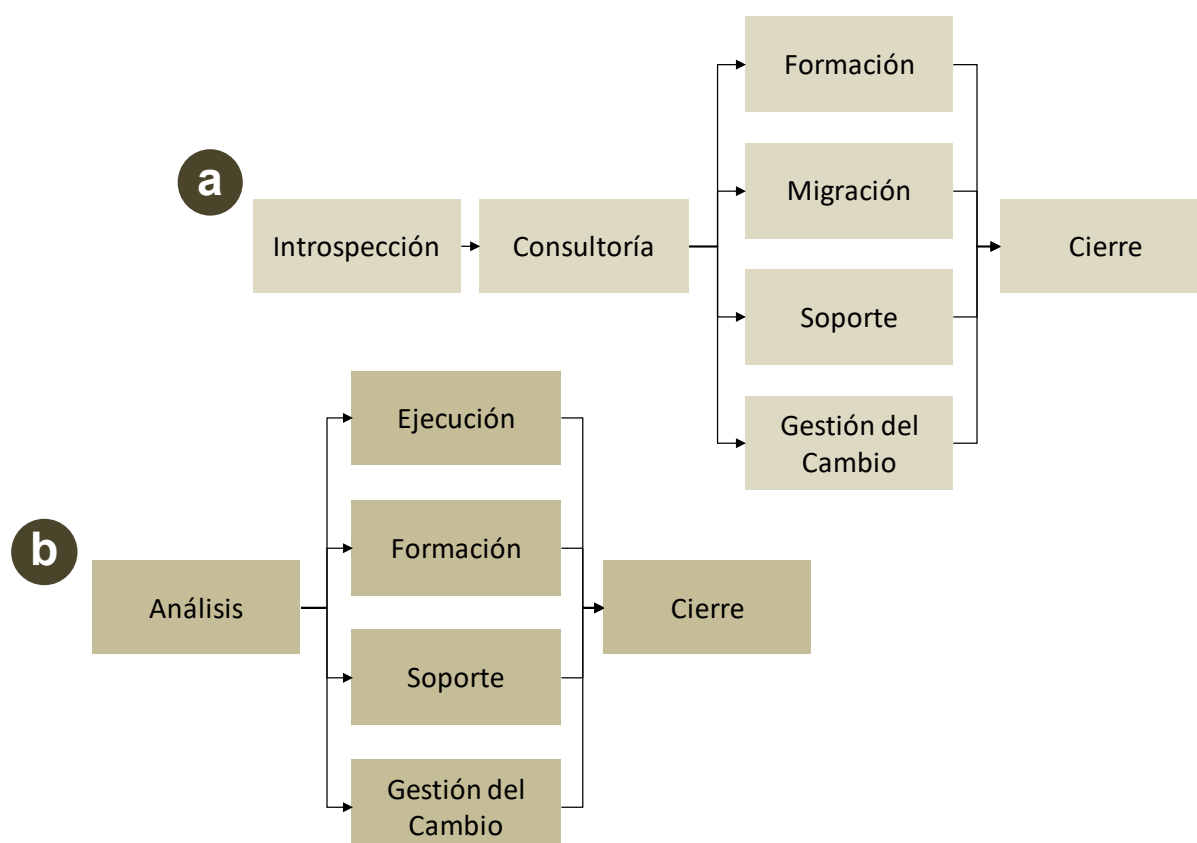


Figura 1. Modelos de migración a software de fuentes abiertas proporcionados por Cenatic. (a) Entornos de escritorio y (b) sistemas de gestión de bases de datos. Fuente: figura adaptada de wiki.cenatic.es.

- **Interoperation** (interoperabilidad) o **wrapping**: consiste en encapsular la aplicación o algunos de sus componentes para que sean utilizados por otras aplicaciones o infraestructura con tecnología más moderna. Por ejemplo, se puede encapsular con servicios web SOAP una aplicación *legacy* que tenga interfaces que no sean SOAP.
- **Retirada**: una aplicación *legacy* puede dejar de ser necesaria al ser su funcionalidad absorbida por otras aplicaciones o dejar de ser requerida.
- **Migración a la nube**: esta modalidad en realidad engloba en mayor o menor medida a varias de las anteriores, pues se puede tratar, por ejemplo, únicamente de un *re-hosting* a un entorno *cloud*, o una sustitución por una aplicación SaaS. Se ha señalado como caso particular por su popularidad en los últimos años.

2.5 Cenatic – Migración de aplicaciones a software de fuentes abiertas

El **Cenatic** dispone de un espacio web (<http://wiki.cenatic.es/>) para compartir experiencias relativas a la migración de aplicaciones a software de fuentes abiertas (*open source*). En la actualidad proporciona metodologías para dos escenarios de migración:

1. Migraciones de entornos de escritorio a *software* de fuentes abiertas. Proporciona una metodología con un proceso en etapas (Figura 1.a).
2. Migración de Aplicaciones a Sistemas de Gestión de Bases de Datos Libres. Proporciona igualmente documentación y una metodología en etapas (Figura 1.b).

En ambas metodologías se proporciona documentación que incluye una descripción de actividades, roles, procedimientos, métricas, referencias a casos prácticos y herramientas para el análisis de costes entre otros.

3. Gestión de la configuración

Existen múltiples definiciones de la gestión de la configuración. La gestión de la configuración puede entenderse como un proceso de la ingeniería del software que tiene como objetivo la calidad de los productos, mediante el control de los cambios sobre los activos *software* y *hardware*. Para ello mantiene una imagen detallada de la situación de los activos y de sus relaciones a lo largo del tiempo.

Existen diversas metodologías, buenas prácticas o estándares que recogen el proceso de gestión de la configuración. En general, pueden definirse dos aproximaciones a la gestión de la configuración:

1. La visión orientada al **desarrollo** del software, cuyo objetivo es controlar el estado del código y sus modificaciones a lo largo del ciclo de vida de desarrollo. Esta es la aproximación contemplada en Métrica V3.
2. Otra visión orientada al **gobierno** (*IT governance*) de los sistemas de información, o a la gestión de los servicios IT (*ITSM*, *IT Service Management*), en vez de estar centrada únicamente en el desarrollo de sistemas. Esta es la aproximación contemplada en ITIL.

En el presente apartado se proporciona una visión de ambas aproximaciones.

3.1 Métrica v3 – Interfaz GC

La Gestión de la Configuración es una de las cuatro interfaces de la Metodología Métrica v3. Métrica señala como **objetivo** de la interfaz “mantener la integridad de los productos que se obtienen a lo largo del desarrollo de los sistemas de información, garantizando que no se realizan cambios incontrolados y que todos los participantes en el desarrollo del sistema disponen de la versión adecuada de los productos que manejan”.

Contempla como elementos a ser controlados los ejecutables, código fuente, modelos de datos, modelos de procesos, especificaciones de requisitos, pruebas, etc.

Existen tres escenarios fundamentales en los que entra en juego la Gestión de la Configuración, los cuales se presentan a continuación.

3.1.1 Estudio de Viabilidad del Sistema (EVS)

Tras el proceso EVS se obtiene el Plan de Gestión de la Configuración. Durante la actividad EVS3 – Definición de Requisitos del Sistema, tiene lugar la actividad **EVS-GC1 – Definición de los requisitos de Gestión de la Configuración** (véase Figura 2). En esta actividad se definen los requisitos generales relativos a la gestión de la configuración y los procesos de control que se llevarán a cabo como el control de versiones, control de cambios, etc.

La actividad **EVS-GC2 – Establecimiento del plan de Gestión de la Configuración** tiene lugar tras la tarea EVS6 – Selección de la Solución. Durante esta actividad se establece el plan y también se define el entorno tecnológico (herramientas) que se emplearán para llevar a cabo el plan.

El plan, contiene, entre otros elementos, el alcance del mismo, responsabilidades, reglas de versionado, el ciclo de estados de los productos, etc.

Una vez establecido el Plan de Gestión de la Configuración, se irán registrando los productos que se obtengan en los procesos de Análisis (ASI), Diseño (DSI), Construcción (CSI), Implantación y Aceptación del Sistema (IAS) de Información que se hayan determinado en el plan como productos a incluir en el sistema de gestión de configuración.

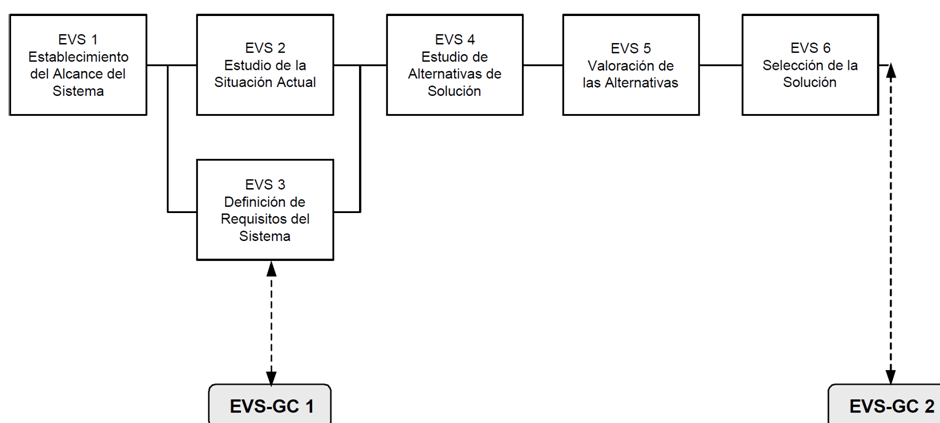


Figura 2. Relación de actividades del proceso EVS con la interfaz de Gestión de la Configuración. Fuente: Métrica v3, © MINHAFP.

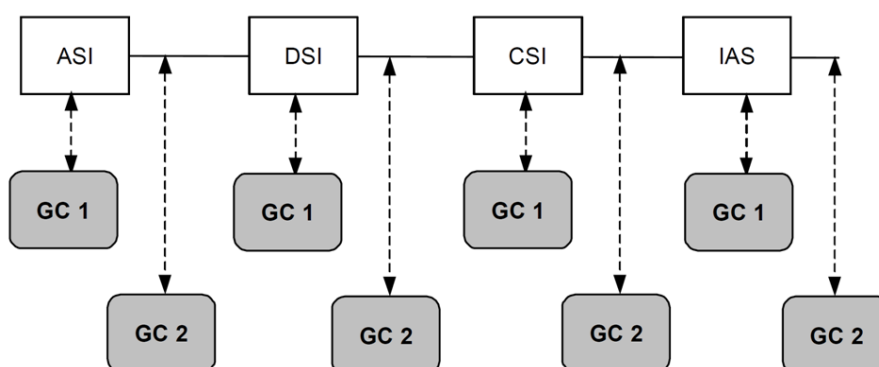


Figura 3. Relación entre los procesos ASI, DSI, CSI e IAS y la interfaz Gestión de la Configuración. Fuente: Métrica v3, © MINHAFP.

3.1.2 Análisis, diseño, construcción e implantación y aceptación del sistema de información

Durante los procesos de ASI, DSI, CSI e IAS el principal cometido de la Gestión de la Configuración es realizar actividades de identificación y registro previstas en el Plan de Gestión de Configuración, consiguiendo así mantener la consistencia entre las distintas versiones de los productos de desarrollo. Según se van generando productos, se van registrando en el sistema de gestión de la configuración con el estado correspondiente. Métrica define los siguientes estados de un producto, como mínimo:

1. En elaboración
2. Finalizado
3. Revisado
4. Aceptado

En la Figura 3 se muestra la relación entre los procesos mencionados y la interfaz Gestión de la Configuración.

La actividad **GC 1 – Identificación y registro de productos** tiene como objeto identificar los productos que se obtienen en los procesos. Les asigna:

- Nombre
- Código de versión
- Estado que indicará la situación en que se encuentran dentro de su proceso de elaboración
- Localización en el sistema de gestión de la configuración

La actividad **GC 2 - Identificación y registro del producto global** se realiza al finalizar los procesos correspondientes, y tiene como finalidad identificar y registrar en el sistema de gestión de la configuración los productos globales que se obtienen a lo largo del desarrollo de los procesos.

3.1.3 Mantenimiento del Sistema de Información (MSI)

El objetivo de la Gestión de la configuración en el proceso MSI es el mismo que en el resto de procesos anteriores: mantener la integridad del sistema y de la información acerca del mismo. En este caso, se velará por la integridad en el mantenimiento correctivo y evolutivo. Si se realiza una gestión de la configuración, se facilitará la resolución de incidencias, identificación de causas, etc.

En este caso se contempla una única actividad de la interfaz: **MSI-GC 1 – Registro del cambio en el sistema de gestión de la configuración**. Esta actividad, una vez realizados los cambios pertinentes en el proceso MSI, registra la nueva versión del producto modificado y la nueva versión del sistema o sistemas impactados por los cambios.

3.2 ITIL – Service and Asset Configuration Management

El proceso ITIL relacionado con Gestión de la Configuración se denomina Service Asset and Configuration Management (SACM), ubicado en el libro Service Transition.

Tiene como objetivo que todos los activos necesarios para proporcionar servicios estén controlados, y que exista información precisa y fiable cuando y donde sea necesaria.

Define los **Configuration Items (CIs)** como todos los **activos de servicio (service assets)** que necesitan ser gestionados para proporcionar un servicio. Un activo de servicio es un recurso o capacidad que contribuye a la entrega de un servicio. Distingue específicamente entre **activos de servicio** y CIs en la medida que los CIs pueden ser claramente identificados y gestionados individualmente y los activos no (por ejemplo, la información almacenada en un servidor, si no está en el alcance de la Gestión de la Configuración, es un activo pero no un CI).

3.2.1 Elementos clave de SACM

Además de la definición de CIs y *service assets*, proporciona las siguientes definiciones:

- **Configuration record** (registro de configuración): es un conjunto de atributos y relaciones acerca de un CI. Los *configuration records* se almacenan en la CMDB (Configuration Management Database) y se gestionan con un CMS (Configuration Management System).
- **Service Knowledge Management System (SKMS)**: es un conjunto de herramientas y bases de datos que se utilizan para gestionar el conocimiento, información y datos. Incluye tanto sistemas de información como bases de datos. Permite recopilar, almacenar, gestionar, actualizar, analizar y presentar el conocimiento, información y datos para la gestión del ciclo de vida de un servicio. El proceso SACM no es responsable de la gestión del SKMS, sino que será propietario de algunos CIs, pero otros procesos serán responsables de otros CIs.
- **Configuration Management System (CMS)**: es un conjunto de herramientas, datos e información, parte del SKMS, en el que se almacenan los *configuration records* de todos los CIs que se encuentren bajo el alcance del proceso de SACM. Algunos de los CIs tendrán información relacionada que no se encuentre almacenada en el CMS, como por ejemplo SLAs o contratos. Estos CIs se almacenan en el SKMS, pero otros CIs como servidores o personas no se almacenan en el SKMS (como es de suponer). Los *configuration records* se almacenan en CMDBs.
- **Configuration Management Database (CMDB)**: es una base de datos en donde se almacenan *configuration records*. Un CMS puede contener una o varias CMDBs.
- **Configuration baseline** (línea de base de configuración): es la configuración de un servicio, producto o infraestructura que ha sido revisada y acordada formalmente, que sirve a partir de ese momento como base para actividades futuras.
- **Snapshot** (captura instantánea): es el estado actual de un CI o de un entorno. No requiere estar formalmente aprobado.
- **Definitive Media Library (DML, Biblioteca de medios definitivos)**: es una biblioteca segura (tanto física como virtual) en donde se almacenan las versiones autorizadas de todos los activos software de la organización. Contiene software comprado y desarrollado en la organización.

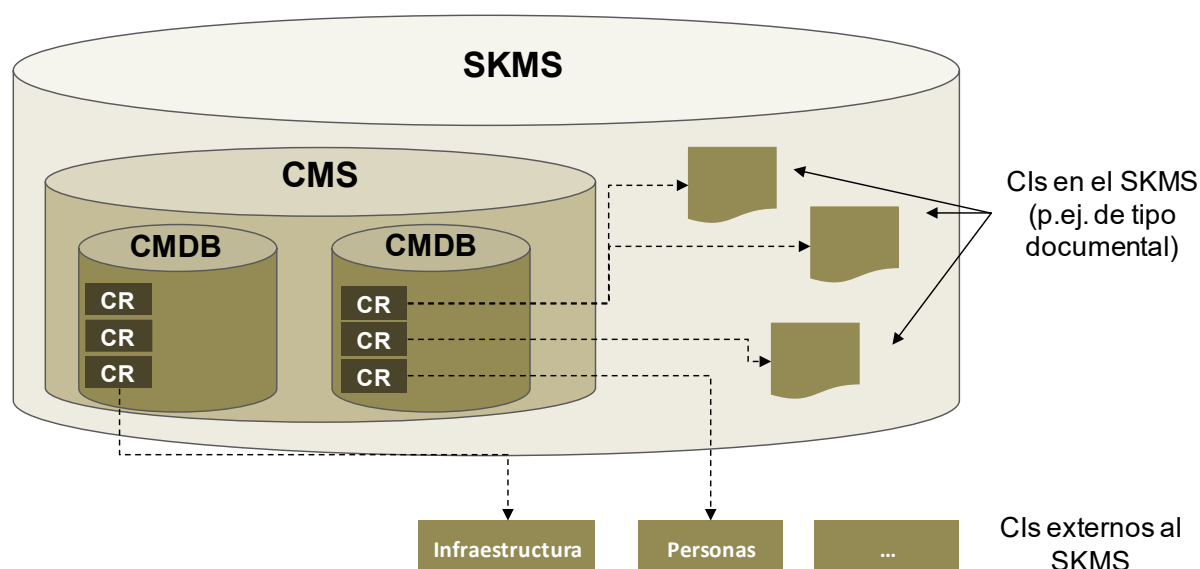


Figura 4. Esquema de relación entre el SKMS, CMS y CMDB.

El proceso de SACM proporciona un *configuration model*, o modelo de configuración que relaciona servicios, activos e infraestructura. Este modelo permite:

1. Evaluar el impacto de incidencias y problemas
2. Evaluar el impacto de cambios propuestos
3. Planificar y diseñar cambios en los servicios o nuevos servicios
4. Planificar actualizaciones tecnológicas
5. Planificar despliegues y migraciones de activos de servicio
6. Optimizar costes

3.2.2 El proceso de SACM

ITIL contempla un modelo de proceso de SACM con 5 actividades clave, aunque señala que cada organización deberá determinar la aproximación más adecuada a sus necesidades. Las 5 actividades son las siguientes:

1. **Gestión y planificación:** define el contexto y propósito del proceso de SACM, alcance, requerimientos, estándares aplicables, organización, herramientas, etc.
2. **Identificación de la configuración:** definir el modelo para identificar activos, roles y responsabilidades, ubicar su información en la CMDB, etc.
3. **Control de la configuración:** asegurar que existen mecanismos de control y mantener registro de los cambios en Cls.
4. **Control de estado y reporting:** asegurar que la información de configuración se almacena de acuerdo al avance de los Cls en su ciclo de vida. Proporcionar datos e informes relativos a los Cls
5. **Verificación y auditoría:** asegurar que los *baselines* documentados son consistentes con el entorno real. Verificar que los Cls documentados existen en el estado en el que figuran. Asegurar que antes de aplicar cambios están correctamente documentados.

3.3 Gestión de la configuración en estándares y metodologías

Existen diversos estándares y metodologías relativos a la gestión de la configuración, muchos de ellos nacionales o para ámbitos específicos. A continuación se proporcionan algunos estándares y metodologías generales relativos a la gestión de la configuración:

- **CMMI – Configuration Management.** Configuration Management (CM) es un área de proceso en el nivel de madurez 2 de CMMI–SCV (CMMI para servicios). Cuenta con tres objetivos: establecer líneas de base, monitorizar y controlar cambios y establecer integridad.
- **ISO 10007:2017 Quality management - Guidelines for configuration management.** Define roles, responsabilidades, y un proceso de gestión de la configuración. Es un documento guía, por lo que no existe certificación en ISO 10007.

- **828-2012 IEEE Standard for Configuration Management in Systems and Software Engineering.** Contempla 7 procesos, denominados *lower level processes*, para la gestión de la configuración: planificación, gestión, identificación de la configuración, control de cambios, control de estados, auditoría, y gestión de despliegues. Utiliza terminología común con ITIL como CI, CMS, CMDB, etc.

4. Gestión de versiones

Al igual que sucede con la Gestión de la Configuración, existen dos aproximaciones a la gestión de versiones: la orientada al desarrollo y la orientada al gobierno de sistemas o gestión del servicio IT (relacionada con los conceptos ITSM e ITIL). En ambos casos la gestión de versiones está relacionada con la puesta en producción de código nuevo o modificado, o de un servicio (en la visión ITSM), si bien la aproximación a la misma es algo diferente.

El término **versiones** se puede asociar en inglés con el término “**release**”, traducido frecuentemente como **entrega**, o con el término “**version**” (traducido como **versión**). Con el fin de asegurar que se contemplan todas las posibilidades, se abordan las dos aproximaciones en este tema (*release* y *version*).

La gestión de versiones (*version control*) suele asociarse al control de los cambios en el código durante el proceso de desarrollo, mientras que la gestión de entregas (*release management*) suele referirse a la prueba, empaquetado y puesta en producción de código.

4.1 ITIL – Gestión de entregas y despliegues

El proceso de gestión de entregas y despliegues en ITIL (*release and deployment management*) pertenece al libro Service Transition. Tiene como finalidad la planificación y control del desarrollo, pruebas y despliegue de versiones/entregas, así como la entrega de la nueva funcionalidad requerida por el negocio protegiendo la integridad de los servicios existentes.

ITIL **define una versión (release)** como uno o varios cambios realizados sobre un servicio IT que son construidos, probados y desplegados conjuntamente. Una versión puede contener cambios a hardware, software, documentación, procesos u otros componentes.

Los objetivos del proceso son los siguientes:

- Definir y acordar planes de despliegue.
- Crear y probar paquetes (conjuntos) de pruebas.
- Asegurar la integridad de un paquete de versión durante el despliegue (es decir, de los activos software que se pasan a producción).
- Desplegar los paquetes de versión y asegurar su trazabilidad.
- Registrar desviaciones, riesgos y eventos relacionados con el nuevo servicio desplegado que requieren acciones.
- Asegurar que hay un proceso de transferencia del conocimiento.

Contempla dos **opciones para el despliegue**. El modo “**big bang**” se refiere al despliegue completo de la modificación realizada a todas las unidades usuarias. Por otro lado, el modo **faseado** o **incremental** contempla el despliegue gradual a diferentes áreas usuarias (aunque también esta estrategia suele emplearse para desplegar modificaciones de forma incremental a todos los usuarios en lugar de todas al mismo tiempo).

Distingue entre despliegues “**push**”, en los que los cambios se propagan desde el servidor de versiones a los equipos usuarios, y despliegues “**pull**”, en los que los usuarios descargan la nueva versión cuando se va a utilizar o cuando, por ejemplo, reinician sus equipos.

El proceso de gestión de versiones y despliegues se define en **4 fases**:

1. Planificación de versiones y despliegues
2. Construcción y pruebas de la versión
3. Despliegue
4. Revisión y cierre

4.2 Gestión de versiones y *releases* en el desarrollo *software*

La gestión de versiones en el ámbito del desarrollo *software* se refiere al conjunto de actividades relativas a gestionar, planificar y controlar las modificaciones de código a través de las diferentes etapas y entornos (desarrollo, integración, pre-producción, etc., tal como se describirá más adelante en el presente tema), incluyendo pruebas y, especialmente, el despliegue de versiones en el entorno productivo.

Es una actividad que adquiere especial relevancia en la medida que los desarrollos se realicen en equipos distribuidos, existan cambios frecuentes o deban gestionarse cambios realizados por varios proyectos en paralelo (*releases* múltiples).

4.2.1 Terminología común en la gestión de versiones

En la gestión de versiones suele emplearse terminología que es de uso común en los proyectos de desarrollo. Cabe destacar los siguientes términos:

- **Repositorio:** se refiere al servidor de versiones y el conjunto de ficheros que conforman el código fuente de la aplicación.
- **Check-out:** consiste en descargar una copia del código fuente a un equipo local desde el servidor de versiones con el fin de, en general, modificarlo.
- **Update:** actualizar la versión local del código con la última versión disponible en el servidor de versiones. De esta manera el desarrollador se asegura que trabaja sobre la versión más reciente.
- **Commit (o check-in):** El término *commit* implica convertir cambios temporales en definitivos, consolidando en el servidor de versiones los cambios realizados en local. En caso de **conflictos** de código, por ejemplo por múltiples modificaciones sobre una misma línea, se deberá resolver, en general, de forma manual.
- **Diff (o Delta):** se refiere a las diferencias entre dos versiones (código añadido, eliminado, modificado, etc.)
- **Delta compression:** consiste en que se almacenen en el repositorio de la herramienta de gestión de versiones únicamente las diferencias entre versiones, en vez de una copia de cada versión, con el fin de reducir la necesidad de almacenamiento.
- **Branch (rama):** consiste en una familia de modificaciones al código fuente independiente de otra, de tal forma que se pueda trabajar de forma aislada en la evolución de una determinada parte del código sin tener que modificar todo el código cada vez que se actualiza. Cuando se crea una nueva rama, se duplica la versión actual del código fuente, y esta se modifica de forma independientemente. Pueden existir múltiples ramas a la vez. Una rama se considerará la rama principal o tronco (*trunk*).
- **Merge:** consiste en fusionar una rama con otra rama (generalmente con la rama principal). Esta labor es compleja para el servidor de versiones pues debe sincronizar múltiples cambios.
- **Fork (bifurcación):** consiste en crear una nueva rama que nunca volverá a ser fusionada con la principal, por ejemplo, para crear un producto con una evolución diferente. A modo de ejemplo, Edubuntu y Linux Mint son *forks* de Ubuntu Linux.
- **Push:** en sistemas de control de versiones distribuido, consiste en enviar el código a otros servidores tras hacer el *commit* en local.
- **Pull:** en sistemas de control de versiones distribuido, consiste en descargar una copia de código de un servidor y sustituir a la local.
- **Fetch:** en sistemas de control de versiones distribuido, consiste en descargar una copia de código de un servidor en el equipo local pero sin sustituir la copia local de código.

4.2.2 Integración continua

La integración continua es una aproximación relativamente reciente, relacionada a menudo con Agile o Extreme Programming, en la que se realizan integraciones de código muy frecuentes (incluso varias veces al día), frente a la aproximación tradicional de integrar el código cuando se han realizado todas las tareas de modificación pendientes o en base a plazos. Se entiende integración de código como el *check-in* del mismo, compilado si procede y ejecución de casos de prueba.

Esta técnica permite obtener versiones de la aplicación con mejoras incrementales en periodos cortos de tiempo y reducir la incertidumbre propia de integraciones de código con un volumen elevado de modificaciones que pueden colisionar o no adecuarse a los casos de prueba previstos.

La integración continua hace uso de herramientas de automatización de la compilación y pruebas como Jenkins o Hudson (las cuales a su vez pueden integrarse con herramientas de gestión de versiones).

4.3 Herramientas para la gestión de la configuración

Desde el punto de vista de las herramientas para la gestión de la configuración, existen dos orientaciones:

1. **Orientación al gobierno TIC o gestión de servicios TI (ITSM).** Herramientas orientadas a la gestión de la configuración de los activos TIC de una organización, desde el punto de vista del *IT governance* o *IT Service Management*. Gestionan fundamentalmente información relativa a los activos TI, como servidores, servicios, documentación, etc. Esta es la orientación proporcionada por ITIL.
2. **Orientación al desarrollo (gestión de versiones).** Herramientas orientadas a la gestión de la configuración y versiones en proyectos de desarrollo. Facilitan la gestión del código, coordinación entre equipos de desarrollo, control de cambios y versiones a lo largo del ciclo de vida de desarrollo. Gestionan fundamentalmente información relativa al ciclo de vida del código fuente y su despliegue. Esta orientación está más relacionada con la proporcionada por MÉTRICA v3.

4.3.1 Herramientas para la gestión de la configuración orientadas al gobierno de los sistemas de información

Esta clase de herramientas están orientadas a la Gestión de los servicios TI (ITSM) y en general forman parte de *suites* de aplicaciones que incluyen más funcionalidad, como gestión de incidencias, gestión de cambios, etc. Las herramientas más populares en este ámbito son de licencia propietaria, entre las que cabe destacar las siguientes:

1. **BMC Remedy.** Una de las más populares, cuenta con funcionalidad completa de CMDB (el módulo de CMDB se denomina BMC Atrium) y otros módulos ITSM adicionales.
2. **EasyVista Service Manager.** La CMDB forma parte de una *suite* ITSM.
3. **ServiceNow.** Igualmente la CMDB como parte de la *suite* de módulos ITSM.

4.3.2 Herramientas para la gestión de versiones y la configuración en proyectos de desarrollo

Este tipo de herramientas permite gestionar el ciclo de vida del código fuente. Permiten el desarrollo en paralelo desde varios equipos de desarrollo, la comparación de versiones, restaurar versiones antiguas, auditar cambios en el código, etc. Estas herramientas no están únicamente orientadas a la gestión de código fuente, aunque éste es uno de sus usos principales. Permite gestionar versiones, en general, de cualquier tipo de fichero, como por ejemplo documentos de texto.

Existen dos aproximaciones principales en la arquitectura de las herramientas de gestión de versiones. La **gestión centralizada, o modelo cliente-servidor**, almacena el código en un servidor central del cual los clientes descargan una copia local (*check-out*), trabajan sobre ella, y la suben de nuevo al servidor (*check-in*). Frente a esta aproximación, el **modelo distribuido** es más reciente y clona el repositorio en todos los equipos de los desarrolladores (todo el historial de cambios y metadatos asociados). Esto permite aplicar cambios rápidamente en local, y una vez que se han realizado todos, publicarlos al resto de repositorios.

A continuación se proporciona una breve descripción de algunas herramientas populares de gestión de versiones.

1. **CVS – Concurrent Versions System.** Escrito en C y con licencia GPL. Herramienta de gestión de versiones en modelo cliente-servidor.

2. **SVN – Apache Subversion.** Escrito en C y con licencia Apache. Más moderno que CVS, adopta también un modelo cliente-servidor.
3. **IBM Rational Clear Case.** Licencia propietaria, modelo cliente-servidor.
4. **Perforce Helix.** Licencia propietaria, modelo cliente-servidor.
5. **Git.** Escrito en múltiples lenguajes, licencia GPL y LGPL. Es más reciente que los dos anteriores y goza también de gran popularidad. Es un sistema distribuido.
6. **Mercurial.** Escrito en Python y C, licencia GPL. Representa otro ejemplo popular de gestión de versiones distribuida, al igual que Git.
7. **GNU Bazaar.** Escrito en Python y C, licencia GPL. Permite modo distribuido y modo cliente-servidor.

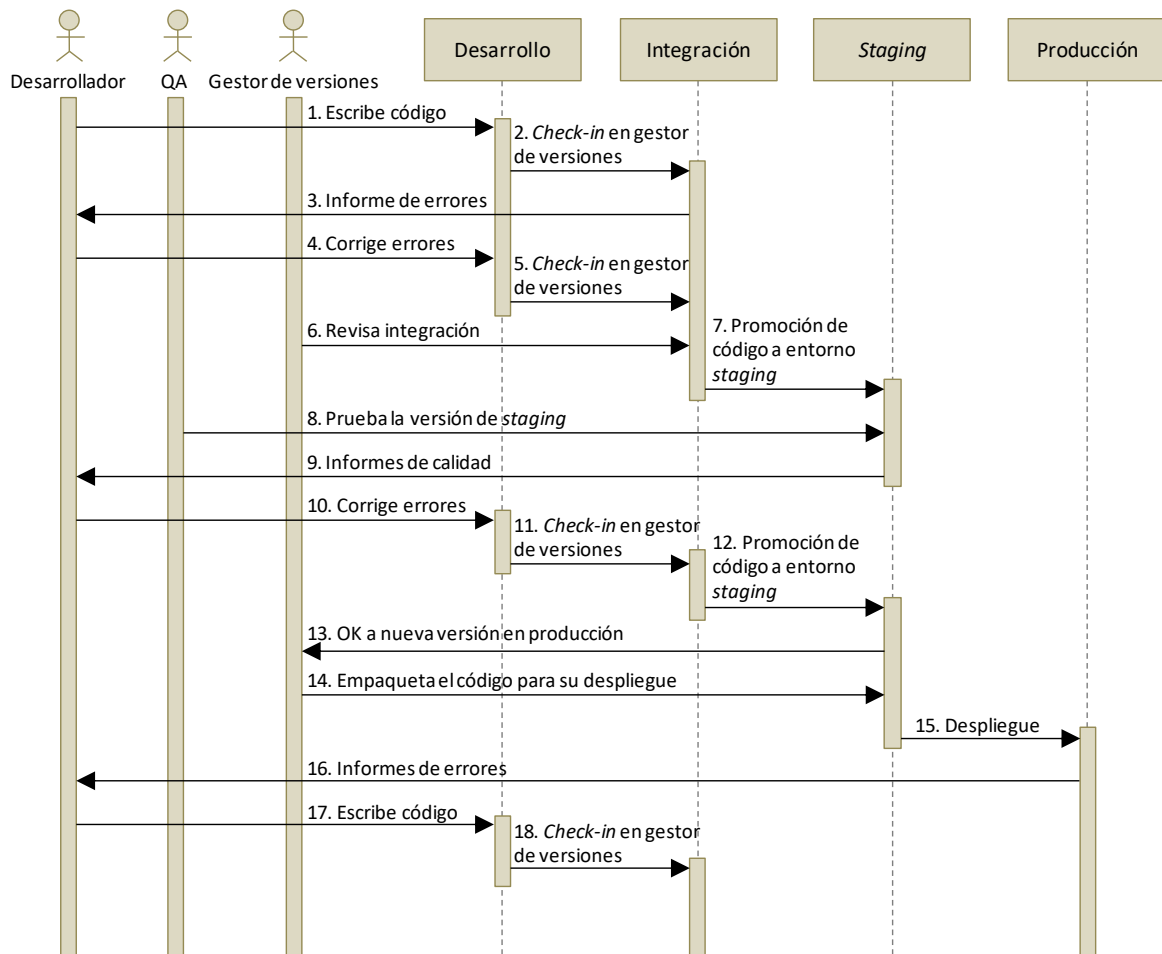


Figura 5. Ejemplo de promoción del código fuente entre entornos y roles implicados.

5. Gestión de Entornos

La gestión de entornos da soporte al ciclo de vida de desarrollo del software, proporcionando diferentes entornos de ejecución de la aplicación, adaptados a cada una de las fases del ciclo. Así, existirán diferentes entornos según la finalidad de la fase. En términos generales, suelen distinguirse los siguientes entornos:

1. **Entorno local:** equipo local del desarrollador donde trabaja en el código fuente de la aplicación.
2. **Entorno de desarrollo:** entorno de trabajo del desarrollador o equipo de desarrollo. Puede ser igualmente el entorno local. En este entorno se ejecutarían las pruebas unitarias.
3. **Entorno de integración:** entorno en el que se realiza el *merge* de las distintas copias del código en la que trabajan los desarrolladores. Permite realizar pruebas de integración.
4. **Entorno de pruebas:** entorno con características más parecidas al de producción. Orientado a la ejecución de pruebas extremo a extremo, pruebas de interfaces, aseguramiento de la calidad, etc.
5. **Entorno de pre-producción** (también denominado *staging*): orientado en general a la realización de pruebas por parte de usuarios. Debe ser lo más similar posible al de producción con el fin de reproducir fielmente el comportamiento de la aplicación en el entorno de producción.
6. **Entorno de producción:** entorno en el que se ejecuta la aplicación.

Según la organización y el tipo de desarrollo, se podrá simplificar el número de entornos (por ejemplo, unificar los entornos de desarrollo, integración y pruebas en el caso de desarrollos sencillos) o hacerlo más complejo. En la Figura 5 se proporciona un ejemplo de ciclo de vida de desarrollo, señalando las promociones de código entre entornos.

En la actualidad no existe una metodología o estándar popular que proporcione pautas para la gestión de entornos de forma particular, aunque esta estará sustentada por otros procesos como la gestión de la configuración, gestión de versiones, etc. A continuación se proporcionan algunas actividades típicas de la gestión de entornos:

- **Reserva de entornos:** si varias personas o equipos trabajan sobre una misma aplicación o sistema, puede ser necesario establecer turnos para la utilización de un entorno determinado, o aislar espacios del entorno para diferentes equipos. Esto sucede, por ejemplo, en los casos de los entornos de pruebas, cuando deben realizarse diferentes tipologías de pruebas por parte de equipos diferentes.
- **Limpieza de entornos:** especialmente en los entornos de desarrollo, se suelen crear elementos temporales fruto del desarrollo (código temporal, ficheros de prueba temporales, etc.). Es importante mantener los entornos “limpios” con el fin de facilitar su uso y hacerlos sostenibles.
- **Coherencia entre entornos:** los cambios en el entorno de producción deberán quedar reflejados en los entornos no productivos (en la medida que resulte aplicable), con el fin de asegurar que los desarrolladores, los equipos de prueba y otros actores cuentan con una imagen fiable del entorno de producción (esto es especialmente relevante en el entorno de pre-producción o *staging*).
- **Gestión de la capacidad de los entornos:** deberá asegurarse que los entornos no productivos cuentan con la capacidad de proceso y recursos suficientes para realizar las pruebas que sean necesarias.

RESUMEN ESQUEMÁTICO

1. Migración de aplicaciones

- 1.1. Ajuste dimensional
- 1.2. Downsizing, Upsizing y Rightsizing
- 1.3. Obsolescencia de aplicaciones y migración
- 1.4. Tipos de migración de aplicaciones: *refronting*, *replacement*, *rehosting*, *rearchitecting*, *interoperation*, retirada, migración a la nube.
- 1.5. Cenatic – migración de aplicaciones a software de fuentes abiertas: entornos de escritorio y sistemas de gestión de bases de datos.

2. Gestión de la configuración: visión orientada al desarrollo y visión orientada al gobierno IT.

- 2.1. Métrica v3 – interfaz GC
 - 2.1.1. Estudio de viabilidad del sistema (EVS): EVS-GC1 y EVS-GC2
 - 2.1.2. Análisis, diseño, construcción e implantación y aceptación del sistema de información: GC1 y GC2
 - 2.1.3. Mantenimiento del sistema de información (MSI): MSI-GC1
- 2.2. ITIL – Service and Asset Configuration Management
 - 2.2.1. Elementos clave de SACM: CR, SKMS, CMS, CMDB, CI, DML, etc.
 - 2.2.2. Proceso de SACM
- 2.3. Gestión de la configuración en estándares y metodologías: CMMI-CM, ISO 10007, IEEE 828-2012.

3. Gestión de versiones

- 3.1. ITIL – gestión de entregas y despliegues
- 3.2. Gestión de versiones y *releases* en el desarrollo software
 - 3.2.1. Terminología común en la gestión de versiones: *check-out*, *check-in*, *update*, *diff*, *delta*, *branch*, *fork*, *pull*, *push*, etc.
 - 3.2.2. Integración continua
- 3.3. Herramientas para la gestión de la configuración
 - 3.3.1. Herramientas para la gestión de la configuración orientadas al gobierno de los sistemas de información: BMC Remedy, EasyVista Service Manager, ServiceNow.
 - 3.3.2. Herramientas para la gestión de versiones y la configuración en proyectos de desarrollo: SVN, IBM Rational, Git, Mercurial, etc.

4. Gestión de entornos

- 4.1. Entorno local
- 4.2. Entorno de desarrollo
- 4.3. Entorno de integración
- 4.4. Entorno de pruebas
- 4.5. Entorno de pre-producción
- 4.6. Entorno de producción
- 4.7. Actividades: reserva de entornos, limpieza, coherencia, gestión de la capacidad.

PREGUNTAS DE TEST

- 1) Con respecto al ajuste dimensional, el *downsizing*:
 - a) Es una técnica reciente, surgida a partir del crecimiento del *IoT* (Internet of Things o Internet de las Cosas), que está relacionada con el paso de entornos cliente-servidor a entornos de computación distribuida.
 - b) Es una técnica orientada al cambio de arquitecturas cliente-servidor, relacionada con el paso de clientes pesados (que requieren la instalación de un aplicativo en el puesto de usuario) a clientes ligeros (accesibles vía web).
 - c) Es conmutativo con el *upsizing*, es decir, si sobre un sistema se aplica *downsizing* y posteriormente *upsizing*, el sistema resultante es equivalente al original.
 - d) Es un término relacionado con el paso de arquitecturas *mainframe* a entornos con múltiples servidores.

- 2) En lo relativo a la obsolescencia de *software*, el concepto End-of-Life (EOL):
 - a) Indica la versión del producto *software* a partir de la cual el proveedor del mismo deja de proporcionar soporte a sus usuarios en lo relativo a ese producto en particular.
 - b) Proporciona las características que tendrá el producto *software* cuando se encuentre funcionalmente maduro, una vez desarrollados todos los requisitos sobre el mismo.
 - c) Debe ser tenido en cuenta por las organizaciones usuarias de un producto *software* con el fin de adelantarse a la posible finalización del soporte por parte del proveedor del *software*.
 - d) Indica la fecha a partir de la cual el proveedor de un producto *software* deja de comercializar un determinado producto pero no cuándo deja de proporcionar soporte al mismo.

- 3) Indique entre las siguientes opciones cuál no se corresponde con un estándar relativo a la Gestión de la Configuración:
 - a) ISO 10007:2017 Quality management - Guidelines for configuration management.
 - b) IEEE 828-2012 IEEE Standard for Configuration Management in Systems and Software Engineering.
 - c) Ninguno de los anteriores se corresponde con un estándar relativo a la Gestión de la Configuración.
 - d) Las respuestas (a) y (b) se corresponden con un estándar relativo a la Gestión de la configuración.

- 4) Identifique la respuesta FALSA entre las siguientes opciones. La Gestión de la Configuración:
 - a) Proporciona requisitos a los equipos de desarrollo relativos a la medida en que una aplicación y sus componentes deben ser parametrizables a lo largo de su ciclo de vida (por ejemplo, a través de ficheros separados por comas o CSV).
 - b) Es una interfaz (GC) descrita en Métrica v3. El Plan de Gestión de la Configuración se define durante el proceso de Estudio de Viabilidad del Sistema.
 - c) Está contemplada en ITIL, fundamentalmente dentro del libro Service Transition (Transición del Servicio).
 - d) Es un proceso contemplado dentro del modelo CMMI-DEV, denominado Configuration Management.

- 5) En el ámbito de la gestión de versiones, identifique la respuesta CORRECTA:
 - a) Concurrent Versions System y Subversion son dos herramientas de gestión de versiones posteriores a 1985.
 - b) Una rama (*branch*) pasa a ser una bifurcación (*fork*) en el momento en el que se crea otra rama paralela partiendo del mismo tronco (*trunk*).
 - c) Un desarrollador que realice *check-out* del código existente en el repositorio de versiones debe necesariamente realizar un *check-in* posterior antes de realizar otro *check-out*.

- d) La compresión delta (*delta compression*) es una característica de algunas herramientas de gestión de versiones orientada a comprimir la información relativa a los cambios (*deltas*) con el fin de poder almacenar una visión completa (*snapshot*) de cada versión intermedia.
- 6) Señale la afirmación CORRECTA. Usted utiliza en un intérprete de comandos el comando `diff` de Subversion para comparar un fichero A y un fichero B, pertenecientes al código fuente de una aplicación:
- a) Si el fichero A y el fichero B son idénticos, el tamaño de la salida en el intérprete de comandos será necesariamente mayor que si A y B son diferentes.
 - b) Si el fichero A y el fichero B son idénticos, el tamaño de la salida en el intérprete de comandos será necesariamente inferior que si A y B son diferentes.
 - c) Si B se corresponde con un *branch* (rama) de A y A es el *trunk*, el tamaño de la salida será necesariamente superior que si B se corresponde con una nueva versión del *trunk* A.
 - d) Si B se corresponde con un *branch* (rama) de A y A es el *trunk*, el tamaño de la salida será necesariamente inferior que si B se corresponde con una nueva versión del *trunk* A.
- 7) En el marco de la modernización de los servicios de TI, se le encarga que implemente una solución de gestión del servicio que cuente con una CMDB para la Gestión de la Configuración. Dado el entorno presupuestario de su organización, se le solicita que la solución sea de fuentes abiertas sin costes de licencia. Indique la solución que recomendaría:
- a) BMC Remedy.
 - b) ServiceNow IT Service Management.
 - c) EasyVista.
 - d) Ninguna de las anteriores.
- 8) Como responsable del desarrollo de un nuevo aplicativo, se le solicita que proponga la utilización de una herramienta de gestión de versiones distribuida. Dado este requerimiento, indique de entre las siguientes cuáles serían herramientas candidatas: (1) CVS – Concurrent Versions System, (2) SVN – Subversion, (3) Git, (4) Mercurial, (5) Bazaar.
- a) 1, 2, 3, 4, 5
 - b) 2, 3, 4, 5
 - c) 3, 4, 5
 - d) 3, 4
- 9) De acuerdo a la información que se proporciona acerca de la Gestión de la Configuración en el libro ITIL Service Transition, indique la respuesta correcta:
- a) El SKMS (Service Knowledge Management System) contiene un CMS (Configuration Management System), el cual puede contener una o más CMDBs (Configuration Management Database).
 - b) El CMS (Configuration Management System) contiene un SKMS (Service Knowledge Management System), el cual puede contener una o más CMDBs (Configuration Management Database).
 - c) El SKMS (Service Knowledge Management System) y el CMS (Configuration Management System) son equivalentes si el CMS contiene únicamente una CMDB (Configuration Management Database).
 - d) La CMDB contiene Configuration Items (CIs) que almacenan información sobre Configuration Records.

- 10) El CENATIC pone a disposición del público en su sitio web herramientas para la migración a fuentes abiertas, indique entre las siguientes cuáles se encuentran disponibles en la actualidad: (A) Migración de entornos de escritorio a software de fuentes abiertas, (B) Migración de servidores web a software de fuentes abiertas, (C) Migración de sistemas de gestión documental a gestores documentales libres, (D) Migración de aplicaciones a sistemas de gestión de bases de datos libres, (E) Migración de aplicaciones a servidores de aplicaciones libres.
- a) A, B, C
 - b) A, B
 - c) A, E
 - d) A, D

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	d
2	c (aclaración: la respuesta "a" es incorrecta porque EOL no se refiere a la versión del software en particular)
3	d
4	a
5	a
6	b
7	d
8	c
9	a
10	d

BIBLIOGRAFÍA BÁSICA

1. CENATIC, *Foro de Intercambio de Experiencias en Migración a Fuentes Abiertas para las Administraciones Públicas*. <http://wiki.cenatic.es/>
2. OGC, *ITIL Service Transition*. The Stationery Office, 2011.
3. Ministerio de Administraciones Públicas. *Gestión de la Configuración*. Serie Métrica v.3. http://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html
4. IEEE, *828-2012 - IEEE Standard for Configuration Management in Systems and Software Engineering*. IEEE Computer Society. <https://standards.ieee.org/findstds/standard/828-2012.html>.
5. ISO, *ISO 10007:2017 Quality management -- Guidelines for configuration management*. <https://www.iso.org/standard/70400.html>.

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. J. M. Quigley K. L. Robertson. *Configuration Management: Theory, Practice, and Application*, CRC Press, 2015.
2. A. Leon. *Software Configuration Management Handbook*. Artech House, 2015.
3. S. Chacon y B. Straub. *Pro Git*, 2nd edition. Apress, 2014. Disponible online: <https://git-scm.com/book/es/v2>
4. Oracle, *Managing Multiple Environments from Development to Production*. Oracle Help Center. https://docs.oracle.com/cd/B28359_01/owb.111/b31280/configs_11.htm